PostgreSQL (latest)
Home: https://www.postgresql.org/docs/current/index.html
Glossary: https://www.postgresql.org/docs/current/glossary.html
Schema: https://www.postgresql.org/docs/current/information-schema.html

PostGIS: http://postgis.net/

PostgreSQL
Post-Ingres SQL.
Relational database.

Database cluster
A single instance of a **PostgreSQL server**.
The cluster contains:
- server configs
- all database metadata
- server process state
etc..

Database
Schema aka namespace
Schema: https://www.postgresql.org/docs/current/ddl-schemas.htm
A **database** is a collection of **schemas**.
A **schema aka namespace** is a collection of **tables**.
Each table is **qualified** by the database and schema
e.g.

mydb.public.mytable	**database** = mydb **schema**=public **table**=mytable
mytable	

e.g. by default, a database has a **public** schema

```
$ createdb mydb
$ psql mydb

> SHOW search_path                                # Default
schema is "public"
   search_path
------------------
 "$user", public

> CREATE TABLE mydb.public.mytable (x INTEGER);         #
All explicit

> CREATE TABLE public.mytable (x INTEGER);              #
Explicit schema

> CREATE TABLE mytable (x INTEGER);                     #
Implicit
```

Defaults:

postgres	default database

information_schema	e.g. information_schema.features
pg_catalog	system schema e.g. pg_catalog.pg_roles table
public	default schema of any database

Installation (macOS Homebrew)
Defaults
host: localhost
port: 5432
database: postgres
user: yuinishizawa (self)
password: none

* Step 1) **Install**

```
## Install
$ brew install postgresql@18
$ brew link postgresql@18

# Start service
$ brew services start postgresql@18
```

* Step 2) Login

```
# Step 4) Login
$ psql postgres

#postgres \conninfo

You are connected to database "postgres" as user
"yuinishizawa" via socket in "/tmp" at port "5432".
```

==========

--- Internals ---

Relation
Tuple
(Data) Page
Dirty page
Bloated page
Glossary: https://www.postgresql.org/docs/current/glossary.html
A **relation** (tables/indexes) is composed of **rows** (tuples)
Data structure for storing relations is called a **page**.
Pages have fixed size. Hence, a table may consist of **multiple** pages.
Pages are loaded into a **shared buffer** for use. e.g. when processing a query
A page in the buffer is **dirty** if it changed but not yet written to **file**.
A page is **bloated** if it has unused or outdated space.

File segment
File for storing pages is called a **file segment**.
File segments have configurable size.

xact
Short for **transaction**

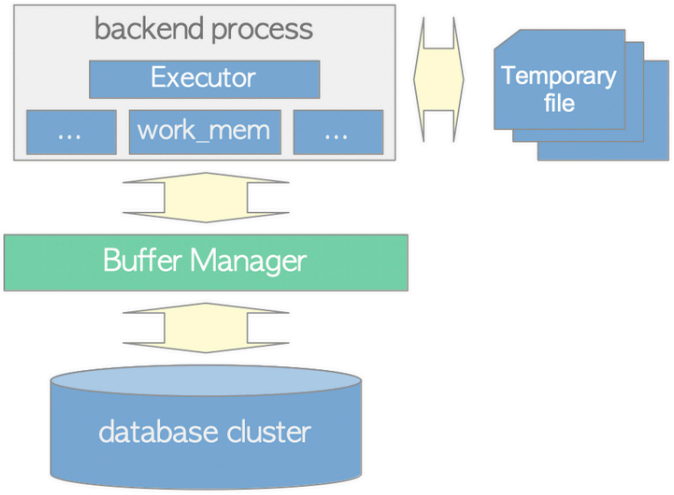
work_mem
temporary file
Query processing: https://www.interdb.jp/pg/pgsql03/index.html
work_mem is memory available for use by a **query**.
A **temporary file** is used if it does not fit in memory.
e.g. hash table of Hash JOIN
e.g. sorting
e.g. MATERIALIZED tables

work_mem is per **operation**, NOT per query.
e.g. if work_mem=4MB and query is

Hash JOIN	max 4MB
Materialize	max 4MB
Sort	max 4MB
	= max 12 MB for whole query

temp_buffer
Memory used to store TEMPORARY data

Shared memory
Shared buffer
Buffer Access strategy
https://www.interdb.jp/pg/pgsql03/01.html
When processing a query, parts of the database data are loaded into **shared memory**.
e.g. page data
e.g. **WAL** records
Shared buffer is the section of shared memory for storing pages.



Multiversion Concurrency Control (MVCC)
13.1 Concurrency Control: https://www.postgresql.org/docs/current/mvcc-intro.html

Multiversion Concurrency Control (MVCC)
Postgres's **concurrency control** system i.e. allow **multiple transactions** in parallel.
Postgres's implementation of **SQL standard transactions**.
Main characteristics are:
- every **tuple (=row)** has a **version**
- every **transaction** has an assigned **version**
  i.e. different transactions may see **different versions**
- **isolation level (standard SQL)**: a transaction **may or may** not see **new COMMITs** depending on the isolation level

WAL
Write Ahead Log (WAL)
WAL record
WAL file/segment
Wiki: https://en.wikipedia.org/wiki/Write-ahead_logging
Reliability and WAL: https://www.postgresql.org/docs/current/wal.html
A **log** describing **fully describing** operations to a database.
i.e. the database can be fully **reconstructed** from a sequence of WAL records
Database analog of **diff patches** in version control?
The smallest unit is called a **WAL record**.
WAL is stored on disk in **WAL segments**.

Checkpoint
The last fully **processed point** in a WAL record sequence.
i.e. WAL records before that point can be discarded.

==========
======

---- OLD ----

Initializing and starting the server
http://www.postgresql.org/docs/9.4/interactive/runtime.html

1. Create a separate postgres user.
2. Initialize the data directory by
   `initdb -D /path/to/data`
3. Start the server by
   `postgres -D /path/to/data`

psql commands
\h Show sql help
\? Show psql help

\l List tables, view

\dg List roles
\dn List schemas
\dt List tables
\dt *.* List All tables in all schemas
\dt earthchem.* List all tables in the earthchem schema
\du List roles
\l List databases

\q quit

CREATE ROLE *user* LOGIN Create a new user with

CREATE EXTENSION postgis

SELECT count(*) FROM *table* Number of rows in table.

SELECT * FROM information_schema.columns